PORTFOLIO

INTRODUCE

오준석

BackEnd Devleoper

Contact.

Email. work.ditto909@passmail.net

Phone. 010-6657-2531

Channel.

Resume. junseokhub.github.io

GitHub. github.com/junseokhub

PROJECTS

Bublitt

2023.09 ~ 2025.05

🔍 프로젝트 개요

OpenAI & RAG 기반 LLM 챗봇 API와 사용자 발화에 대한 주제 추천 기능과 챗봇 통계를 결합한 AI 챗봇 플랫폼

⚙️ 사용 기술 스택

BackEnd : SpringBoot, Nestjs, TypeORM, JPA, Flask(DBSCAN 용)

Database : MySQL, Milvus, Redis, MongoDB

Infra : Kubernetes, BuildKit, ArgoCD, Jenkins, AWS(VPC, Bastion Host, EC2, LoadBalancer), Nginx, Grafana, Prometheus

✅ 주요 성과 및 기술

- **자연어 처리 및 벡터화 기반 주제 추천 시스템 개발**
  - OpenAI Embedding + Milvus/Pinecone 벡터 DB를 활용한 **실시간 유사도 검색 및 주제 추천**
  - **DBSCAN 클러스터링 기반 Fallback 추천 로직** 적용으로 챗봇 응답 신뢰도 **20% 향상**
- **데이터 처리**

    - **Spring Batch와 Spring Scheduler**를 활용한 통계 데이터 자동화 처리
    - PDF 파일의 텍스트를 벡터화 및 임베딩 처리
    - 웹 크롤링 데이터를 **PDF 변환 후 텍스트 기반 임베딩 벡터 DB에 저장**
    - TypeORM 트랜잭션 구조를 JPA 전파 모드로 재설계
- **플랫폼 및 아키텍처 고도화**
    - 백엔드 프레임워크를 **NestJS(TypeORM)에서 Spring Boot(JPA)로 마이그레이션**
    - 검색(Vector), 생성(LLM), 과금(Token), 로깅 프로세스를 비동기 체인으로 연결하여 챗봇 응답 지연 시간(Latency) 최소화 → **전체 응답 시간 30%개선**
    - AWS VPC 내 내부망 구성 및 Bastion Host를 통한 접근 제어로 보안 환경 구축
- **운영 자동화 및 모니터링 체계 확립**
    - **Jenkins CI/CD → ArgoCD + Helm 기반 GitOps 전환**으로 배포 효율성 및 운영 안정성 향상
    - Kubernetes 환경에서 Prometheus와 Grafana를 구축하여 컨테이너 리소스 및 애플리케이션 메트릭을 모니터링
- **데이터 품질 및 테스트 자동화**
    - 사용자 발화와 벡터 데이터의 매칭 정확도를 검증하는 **테스트 모듈 개발 및 자동화 파이프라인 구성**

📂 URL/PDF 문서 관리 및 연동

- URL 및 PDF 문서 메타데이터 수집 및 저장
- 프로젝트별 문서 조회, 제목별 그룹핑 기능 구현
- 중복 체크 및 제목 일괄 수정 지원
- **임베딩 파이프라인과 연계 → 대화 맥락에 실시간 반영**

🔁 CI/CD 구조 개선 핵심

- CI/CD 개선하면서 서버 자체를 Kubernetes환경으로 변경
- Jenkins는 **CI(빌드 및 이미지 푸시)** 전담
- **ArgoCD Image Updater**로 이미지 버전 자동 감지 → Git 매니페스트 업데이트 ArgoCD는 이를 기반으로 **CD(배포 및 상태 모니터링)** 전담 → GitOps 방식 자동화로 **배포 실패율 30% 감소**
- 명확한 역할 분리 및 롤백 가능한 구조 → 유지보수 용이성 향상
- 애플리케이션 배포 시 환경변수를 kubernetes secret을 사용해 docker이미지 빌드 시점이 아닌 런타임 시점에 주입으로 변경

⚠️ 기술 도전 및 한계

- eps (반경) 파라미터와 min_samples (최소 샘플 수) 파라미터 튜닝에 어려움
  - eps 값은 데이터셋의 밀도 분포 변화에 따라 최적 지점을 찾기 어려움 → **일관성 부족**
  - min_samples는 초기 로그 기반으로 산정했지만, 시간이 지나면서 데이터의 밀도 변화를 제대로 반영하지 못해 특정 상황에서는 군집이 너무 많이 생성되거나(노이즈 과다), 반대로 관련 없는 데이터가 하나의 큰 군집으로 병합되는 문제가 발생하여 클러스터링 결과가 불안정 → **노이즈 과다 / 군집 병합 문제 / 결과 불안정**
- 클러스터링을 위해 사용자가 매주 직전 주 데이터에 대한 eps 및 min_samples 범위를 직접 지정해야 함 → **사용자 편의성 및 활용률 저하**
- 마이그레이션
  - I/O 바운드 작업의 지연을 해결하기 위해 CompletableFuture 기반의 비동기/병렬 처리 구조를 도입 → 순차적 처리의 병목을 크게 개선해주었지만, 점점 증가하는 동시성과 더 복잡해지는 비동기 스트림 관리 측면에서는 **CompletableFuture만으로는 한계**
  - Lazy Loadding / Fetch 차이
    - TypeORM은 Lazy Loading이 Promise 기반
    - JPA는 Lazy Loading 시 세션범위가 유지되어야 하며, 지연 로딩 중 session이 닫히면 LazyInitializationException 발생
  - 멀티스레드 환경 동시성 문제
    - TypeORM은 대부분 Promise 기반 호출이나 싱글스레드 환경에서 실행되어 충돌 날일이 적었음
    - DB에서 Lock을 걸지 않아도 동시성 문제가 눈에 잘 보이지 않았음
    - Token 과금 처리, 벡터 DB 업데이트 등에서 동시에 여러 요청이 들어올 때 정합성 깨짐

🛠️ 대안 및 향후 방향

- eps, min_samples
  - 매주 서로 다른 데이터가 입력되면서 데이터의 밀도가 지속적으로 변동하는 구조
  - 이에 따라 eps를 사용하는 DBSCAN의 성능이 일정하지 않을 가능성 → **HDBSCAN 도입** → 계층적 밀도 기반 클러스터링을 수행하여 eps 값을 명시적으로 정하지 않아도 클러스터 효과적으로 식별하고, 밀도가 상이한 클러스터도 유연하게 처리가 가능 할 것으로 예상됨
- **UMAP** 적용 고려 → 차원 축소를 통해 데이터의 구조를 시각적으로 탐색하고 클러스터의 특성을 이해하며, 클러스터링 전 단계에서 노이즈 제거 및 데이터 정제에 활용할 수 있을 것으로 기대됨
- clustering
  - 클러스터링 자동화 → **사용자 편의성 및 활용률 향상**

- 마이그레이션
  - 비동기 처리
    - Embedding, Vector Search 등 **I/O 병목은 기존 CompletableFuture에서 점진적으로 Mono (Spring WebFlux)로 병렬화** 예정
    - 통계/로그/알림 같은 **비동기 후처리는 Kafka/Redis 이벤트 큐 기반**으로 처리하여 확장성과 응답 성능을 모두 확보 예정
  - LazyLoading
    - **Fetch Join**과 **DTO Projection** 전략을 사용하여 필요한 데이터만 미리 조회
    - 서비스 계층에서 트랜잭션 범위를 명확히 지정하여 LazyInitializationException 방지
  - 멀티스레드 환경 동시성 문제
    - Optimistic/Pessimistic Lock + 트랜잭션 설계 + 필요 시 재시도 로직으로 데이터 정합성 확보

🧩 마무리

> Bublitt 프로젝트를 통해 임베딩 기반 군집화 자동화의 한계를 체감했고, 거리 함수, 차원 축소, 파라미터 간 상호작용에 대한 **깊은 기술적 인사이트**를 확보
>
> 다양한 비정형 데이터를 실시간 대화에 반영하는 시스템을 완성하며, **실무 활용성과 확장성**을 모두 갖춘 AI 챗봇 플랫폼을 구축
>
> **TypeORM에서 JPA로 백엔드 마이그레이션**을 수행하며 트랜잭션 전파, Lazy Loading, 멀티스레드 동시성 문제 등 실무적 이슈를 직접 설계,해결

Coconutive

2023.10 ~ 2024.12

🔍 프로젝트 개요

챗봇 통계 솔루션 개발 프로젝트

⚙️ 사용 기술 스택

BackEnd : Nest.js, TypeORM

Database : Redis, MongoDB

✅ 주요 성과 및 기술

- **Raw SQL → TypeORM 전환 (70% 이상)**

    - 주요 SQL 로직을 TypeORM QueryBuilder로 전환하여 쿼리 로직 추상화 및 유지보수성 개선
    - 데이터 접근 구조 단순화 및 **쿼리 성능 최적화**
- **Redis 기반 세션 관리 및 보안 강화**
    - **중복 로그인 세션 문제 32% → 0% 해결**, 세션 관리 로직 개선으로 **보안성과 안정성 강화**
- **LLM 기반 통계 시스템 전환**
    - 기존 **자연어 엔진 기반 통계 기능 → LLM 기반 구조로 재구성**,사용자 발화 및 응답 패턴 분석을 통한 **인사이트 제공 기능 강화**
- **Bublitt 연동 및 전용 통계 대시보드 구축**
    - 기존 사내 플랫폼의 한계를 극복하기 위해 **Bublitt 전용 통계 대시보드** 신규 개발(추후 Bublitt으로 이전 완료)
    - 어드민 관리 및 통계 데이터 시각화 기능 개선
- **MongoDB 기반 자동화 로그 적재 시스템 구축**
    - 기존 수동 로그 파일 관리 방식을 **CronJob으로 MongoDB에 자동화 적재 시스템으로 전환**
    - 텍스트 데이터의 **실시간 검색 및 분석 효율 대폭 향상**
    - 데이터 누락 위험 최소화 및 **운영 안정성 확보**

⚠️ 기술 도전 및 한계

목적: 고객사 별로 모든 환경에서 동작 보장이 어려워 ORM 도입으로 DB 추상화 및 독립성 확보

- RawQuery 의존
    - 각 DB 마다 LIMIT/OFFSET, 문자열, 날짜, JOIN 최적화 문법이 조금씩 다름
    - DB별 쿼리 유지보수 부담 증가
- QueryBuilder 전면 교체의 어려움
    - 다중 JOIN, 동적 조건 필터, 서브쿼리, 집계 함수, 페이징 등 복잡한 쿼리가 너무 많음 → 리팩토링이 아닌 쿼리 재작성 수준
    - 대부분 GET + 복잡 조회 쿼리
- 가독성 저하
    - 오히려 복잡해 질 수록 SQL보다 가독성이 떨어짐
    - 쿼리 구조가 한눈에 안 들어옴

이로 인해 현실적으로 100% 변경이 어렵다고 판단

🛠️ 대안 및 향후 방향

- RawQuery 허용 기준 정의
  - 복잡한 통계/집계 쿼리 → 유지
  - 단순 CRUD 및 동적 필터가 많은 조회 → QueryBuilder
- 남은 RawQuery를 DB 별로 SQL 분리 + Executor
  - 런타임에 현재 DB 종류 확인 후 해당 SQL 로딩
  - 오히려 코드가 더 복잡해 질 위험

🧩 마무리

> 모든 쿼리를 한 번에 바꾸는 것은 기술적 완성도 일 수 있지만, 비즈니스의 연속성을 유지하며 안정적으로 전환하는 것이 엔지니어링의 본질이라고 판단
> RawQuery의 엄격한 관리와 QueryBuilder의 전략적 도입을 병행하며, 최종적으로는 어떤 환경에서도 동작하는 지속 가능한 코드 베이스 구축을 목표해야 함

식약처 코스봇

2024.05 ~ 2025.05

🔍 프로젝트 개요

기존 분류형 챗봇에서 정보 탐색 난이도가 높고, 챗봇의 기계적인 말투로 인해 Bublitt 플랫폼 기반, Upstage RAG(Solar) 도입. 식약처 맞춤형 LLM 챗봇 개발.

⚙️ 사용 기술 스택

Backend : SpringBoot, JPA

Database : MySQL(RDS), Redis

Infra : AWS(VPC, EC2, LoadBalancer), Docker, Jenkins, Nginx

✅ 주요 성과 및 기술

- **RAG(Solar) 아키텍처 도입 및 응답 품질 개선**
  - **RAG(Solar) 도입 전후:**
    - 재질문율 **38% → 21% 감소**
    - Fallback 응답 비율 **24% → 8% 감소**
  - **Rerank 모델 적용**으로 상위 응답률 정확도 **60% → 85% 상승**
- **LLM 통계 및 성능 분석 로직 개발**

    - **Embedding 모델(Upstage / OpenAI)** 별 응답 정확도 비교 테스트 모듈 개발
    - **Vector Search Score** 기반의 응답 품질 평가 체계 구축
    - **기관 맞춤형 통계 로직 및 대시보드** 개발로 **의사결정 지원 및 사용자 인사이트 제공**
- **Bublitt 플랫폼 연동을 통한 통합 어드민 관리 시스템 개발**
    - **Bublitt을 연계**하여 LLM 응답 제어 및 대화 주제/답변 관리 기능 구현
    - **Spring Scheduler와 Spring Batch를 이용해** 시스템 로그 및 사용자 요청 데이터를 주/월 단위로 저장
- **모니터링 및 운영 안정성 강화**
    - **Grafana, Prometheus** 기반의 **실시간 모니터링 및 알림 시스템** 구성
    - **Kubernetes, AWS, Docker, Jenkins, Nginx** 환경에서의 안정적 배포 및 운영 자동화 구현

🧩 마무리

> RAG 기반 LLM 챗봇을 성공적으로 구현하여 **응답 일관성과 정확도 향상에 기여**했으며, 식약처 특화 통계 기능을 통해 **실질적인 정책 결정에 필요한 인사이트 제공**능력 확보.

SK-EcoPlant 사내 챗봇

2023.04 ~ 2025.05

🔍 프로젝트 개요

초기 에코플랜트 사내포털(에피)에서 트위터 기능을 사용해 사내직원들의 불편사항접수 요청이 게시되는 분위기 형성되어 운영자측에 피로감이 존재하여 사내 챗봇을 구축해 부정적인 분위기 탈피하고자 대화형 챗봇이아닌 정보형 챗봇 구축.

⚙️ 사용 기술 스택

BackEnd : Nest.js, TypeORM, GoogleDialogFlowCS

Infra : AWS(CloudFront, EC2, LoadBalancer, S3), Jenkins, Docker

✅ 주요 성과 및 기술

- **백엔드 및 인프라 단독 구축**
    - 프로젝트 전반의 **서버, 인프라, 배포 파이프라인**을 독립적으로 설계 및 구현
    - **Nest.js + TypeORM** 기반 백엔드 아키텍처 구성 및 API 서버 구축

    - DialogFlow 연동을 통해 어드민에서 **Intent / Entity 등록·수정·삭제**가 가능한 **백엔드 API** 개발
- **Google DialogFlow ES 통합 및 관리 기능 개발**
    - 각 대화 시나리오별 **Slot Filling** 적용으로 필요한 정보 자동 수집 및 문맥 기반 응답 강화
    - 접수 데이터를 **DialogFlow SlotFilling 결과값 구글 스프레드 시트에 실시간 업데이트**
- **프론트엔드 챗봇 SDK 배포 자동화**
    - **AWS S3 + CloudFront**를 이용한 정적 리소스 배포 및 자동 업데이트

⚠️ 기술 도전 및 한계

고객사 요청에 의해 스프레드 시트를 활용해 불편접수를 실시간 반영하게끔 했지만, 고객사 측에서 실수로 스프레드시트의 행/열을 삭제 및 수정 했을 시 복구 문제 발생

🛠️ 대안 및 향후 방향

- 스프레드 시트 편집 권환 최소화 및 필수 컬럼은 삭제 못하게 단기적 보호 조치

향후 방향

- 실시간 동기화 (스프레드시트 ↔ 임시 테이블)
    - 즉시 반영 문제를 주기적 폴링으로 해결하려 했으나 데이터 변경이 없는 불필요한 상황에도 동기화가 되어 리소스 낭비
    - 상태 변화 감지 및 이벤트 기반 동기화 필요
- 시트 대신 어드민 페이지 구축
    - 어드민에서 불편접수를 관리

🧩 마무리

> LM 전환 및 버블잇 활용으로 사용자에게 좀 더 정확하고 자연스러운 제공이 필요하다.
> 실시간 동기화와 어드민 페이지 구축을 할 지 고민이 많이 되었는데, 이는 사용자의 편의성이 우선이냐 데이터 정확도와 보안이 우선인지로 구분지어 생각하여 최종적으로는 어드민 페이지를 구축하는 것을 목표로 함

운영 업무

<u>신세계 면세점 상담톡</u>

2023.03 ~ 2023.08

⚙️ 사용 기술 스택

- Vue.js, Express.js, OracleDB, GoogleDialogFlowES

✅ 주요 성과

- 중복 함수 및 조건문 제거로 코드 가독성 및 유지보수성 개선
- 고객사 요청 주요 쿼리문 전달
- 신규 기능 개발 및 유지보수

삼성 닷컴

2022.04 ~ 2023.06

⚙️ 사용 기술 스택

- Vue.js, Node.js, Sequelize, AWS, IBM Watson

✅ 주요 성과

- **서비스 안정성 확보**
  - 3개월 내 **120건 이상의 이슈 신속 해결**로 서비스 안정성 강화
  - 장기 프로젝트 코드 재구조화를 통해 **유지보수성과 시스템 안정성 대폭 향상**
- **보안 강화**
  - 기존 코드의 환경변수 보안 문제 해결: **Docker 배포 시 환경변수 주입 방식** 적용
  - Admin 페이지 **AWS Cognito MFA 적용**으로 인증 보안 강화

2024.12.31부로 서비스 종료